DAGENTS

The DAgents Framework

A reusable framework for deploying one computation agent per source, combining their outputs through a global agent, and producing workload plans for real backend systems.



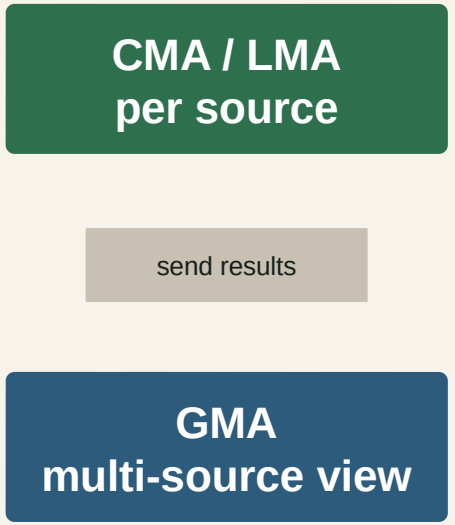
Pradyun Devarakonda
Rajanala Dheeraj (Previously

DEFINITION

Dagents deploys computation agents near data sources, then combines their work globally.

In the codebase, the per-source agent is called LMA (Local Monitoring Agent) or the CMA (Constraint Monitoring Agent). The outputs produced by them are combined and computations on the combined data are then done by the GMA (Global Monitoring Agent).

- One LMA is attached to each source boundary: tenant, database, service, event stream, or environment.
- Each local agent profiles data, partitions work, runs source-level models, and publishes summaries or artifacts.
- The GMA registers local agents and combines their outputs for multi-source computation.
- Core services turn those plans into workflows, model jobs, and Kubernetes workload manifests.



TASK 01 / DATA

Some tasks you can do...

Understand a source before choosing partitions, models, or pipelines.

- **Source profiling:** Turn raw source metadata into a structured DatasetProfile.

TRY IT

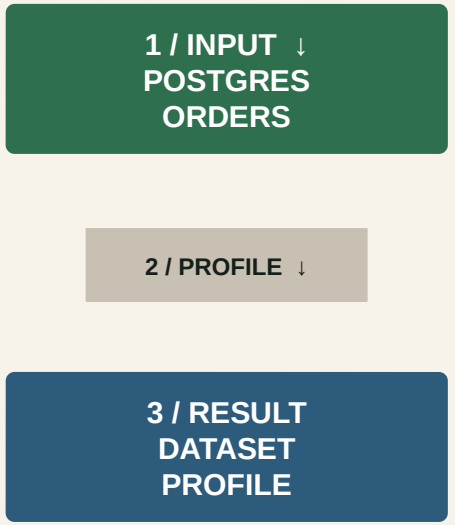
"Profile the Postgres orders table before anomaly detection."

WHAT DAGENTS DOES

- Read the schema and source statistics.
- Find field types, gaps, and cardinality.
- Suggest partitions and suitable model tasks.

RESULT

DatasetProfile plus a data-quality summary



WHY IT MATTERS Better model choices start with a clear picture of the data.

TASK 02 / WORKFLOW

Some tasks you can do...

Turn a workflow description into a checked and ordered execution plan.

- **Pipeline orchestration:** Validate steps and dependencies before runtime execution.

TRY IT

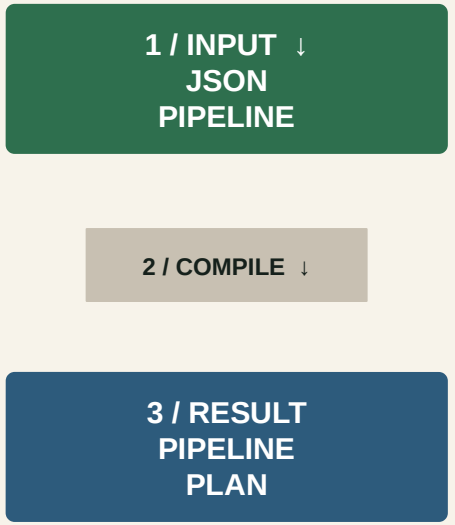
"Run: load orders, filter paid, profile, then detect anomalies."

WHAT DAGENTS DOES

- Check step IDs and input references.
- Reject missing dependencies and cycles.
- Produce a safe topological execution order.

RESULT

Validated pipeline plan plus execution order



WHY IT MATTERS The runtime receives a valid plan instead of guessing step order.

TASK 03 / MODELS

Some tasks you can do...

Select a model from task needs, dataset traits, and available adapters.

- **Model routing:** Centralize model selection and keep fallback choices visible.

TRY IT

"Choose a model for tabular anomaly detection on transaction data."

WHAT DAGENTS DOES

- Match the requested task and dataset profile.
- Check registered model capabilities.
- Rank candidates and choose a packaging mode.

RESULT

Selected model route plus fallback candidates



WHY IT MATTERS Apps ask for a task; the framework owns the model-choice policy.

TASK 04 / DEPLOY

Some tasks you can do...

Convert a typed workload request into deployment-ready Kubernetes YAML.

- **Manifest generation:** Render repeatable deployment material from a WorkloadSpec.

TRY IT

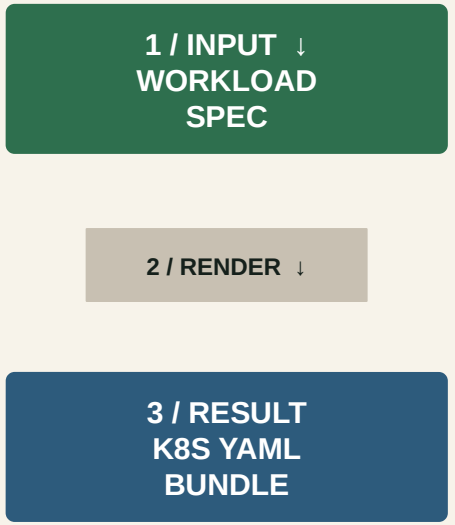
"Deploy lma-orders with two replicas in the dev environment."

WHAT DAGENTS DOES

- Validate image, resources, environment, and ports.
- Apply deterministic naming and labels.
- Render the required Kubernetes resources.

RESULT

KubernetesManifestBundle ready for review or apply



WHY IT MATTERS One workload contract can produce consistent deployment files.

TASK 05 / AGENTS

Some tasks you can do...

Give the GMA a live catalog of local agents, scopes, and capabilities.

- **Agent registration and discovery:** Track which LMA can perform each source-level task.

TRY IT

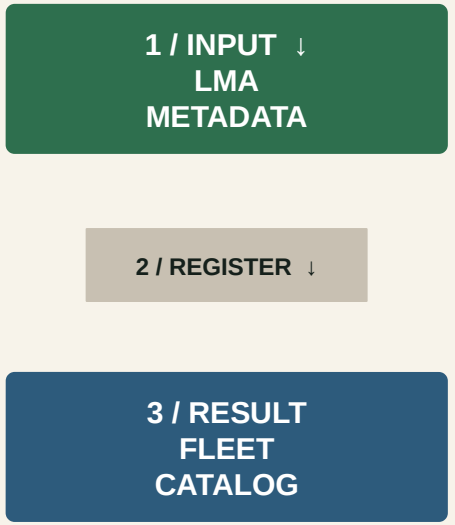
"Register orders for Postgres profiling and forecasting."

WHAT DAGENTS DOES

- Validate identity, scope, version, and capabilities.
- Store the current agent registration.
- Expose the capability through fleet discovery.

RESULT

Discoverable LMA registration and capability record



WHY IT MATTERS The GMA can route work only after it knows what each LMA can do.

TASK 06 / LOCAL

Some tasks you can do...

Run source-scoped work near a tenant, database, service, or event stream.

- **Local source execution:** Keep each computation inside a clear source boundary.

TRY IT

"Run anomaly detection for tenant-a orders."

WHAT DAGENTS DOES

- Resolve the source scope and current bundle.
- Profile, partition, and invoke the selected model.
- Publish a summary and artifact pointer.

RESULT

Source-level run with output metadata



WHY IT MATTERS Local work remains isolated, traceable, and close to its data.

TASK 07 / GLOBAL

Some tasks you can do...

Combine local results when the question spans several sources or tenants.

- **Multi-source assimilation:** Use the GMA for aggregate profiling, correlation, and models.

TRY IT

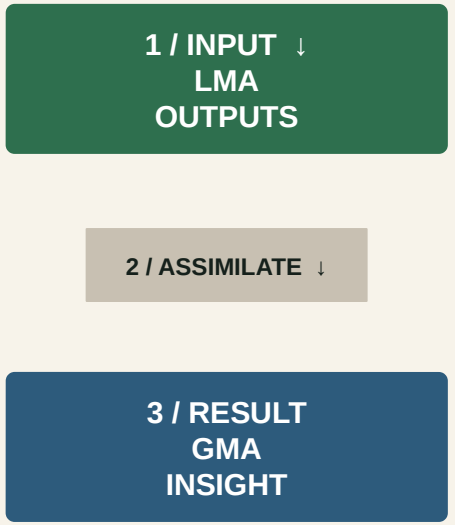
"Combine three sales summaries and find cross-region anomalies."

WHAT DAGENTS DOES

- Collect compatible outputs from registered LMAs.
- Normalize and profile the assimilated data.
- Run aggregate models or correlation rules.

RESULT

Multi-source insight with correlated events



WHY IT MATTERS The GMA turns separate local results into one combined view.

TASK 08 / PLANNING

Some tasks you can do...

Translate computation intent into a typed plan the runtime can execute.

- **Workload compilation:** Resolve task, model, resources, partitions, and outputs early.

TRY IT

"Compile this profiling and anomaly-detection job for runtime."

WHAT DAGENTS DOES

- Validate input, task, model, resources, and output.
- Resolve model routing and partition strategy.
- Emit an explicit execution-ready workload plan.

RESULT

Typed WorkloadPlan with no hidden runtime decisions



WHY IT MATTERS Compilation makes runtime behavior inspectable before execution.

TASK 09 / CONTRACTS

Some tasks you can do...

Plan safe extraction and transformation from a typed source contract.

- **Schema and extraction planning:** Catch missing fields and invalid transforms before data moves.

TRY IT

"Extract order_id, amount, region, and created_at from Postgres."

WHAT DAGENTS DOES

- Validate fields, types, and required values.
- Plan projections, renames, and transforms.
- Return the expected output schema and checks.

RESULT

ExtractionPlan plus output schema and validation report



WHY IT MATTERS A shared data contract removes repeated defensive checks.

TASK 10 / ML

Some tasks you can do...

Coordinate dataset preparation, training, evaluation, and saved artifacts.

- **Model training and inference:** Run the model lifecycle through one service boundary.

TRY IT

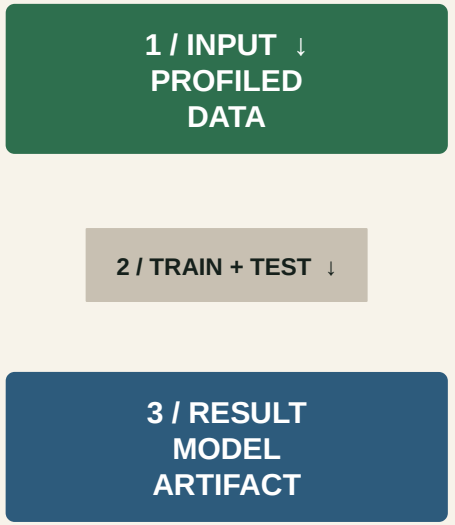
"Train and evaluate an autoencoder on profiled transaction records."

WHAT DAGENTS DOES

- Load and preprocess the selected dataset.
- Train with validation or cross-validation.
- Persist metrics, status, and the model artifact.

RESULT

Model artifact, evaluation metrics, and run status



WHY IT MATTERS DAgents coordinates the lifecycle while Python handles numerical work.

TASK 11 / REUSE

Some tasks you can do...

Register common JSON steps once and reuse them across products and runs.

- **Reusable pipeline steps:** Compose filtering, projection, profiling, and model execution.

TRY IT

"Build: filter, project, profile, then run a model job."

WHAT DAGENTS DOES

- Resolve built-in step implementations.
- Check compatible inputs and outputs.
- Register the named pipeline and run it on JSON.

RESULT

Reusable pipeline with inspectable run history



WHY IT MATTERS Products share workflow behavior without copying orchestration code.

TASK 12 / TRACE

Some tasks you can do...

Keep execution status, summaries, and artifact locations easy to inspect.

- **Run tracking and artifact publishing:** Make every computation auditable after it leaves the caller.

TRY IT

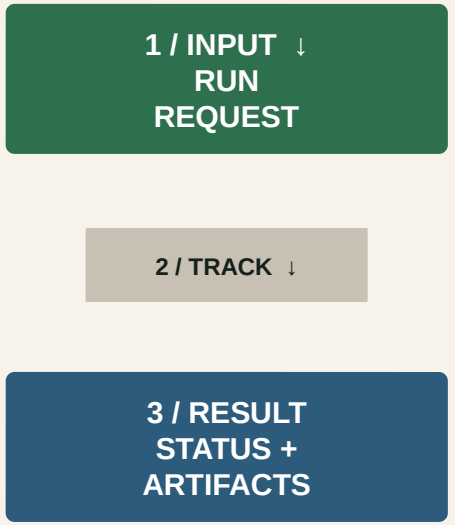
"Show the status and artifacts for run-204."

WHAT DAGENTS DOES

- Create a stable run identifier.
- Record state transitions and summary metadata.
- Publish artifact pointers for later inspection.

RESULT

Auditable run record with status and artifacts



WHY IT MATTERS Every run leaves evidence that another service can inspect.

TASK 13 / OPS

Some tasks you can do...

Connect typed plans to service discovery and deployable workloads.

- **Deployment and service operations:** Inspect topology and generate deployment material for services.

TRY IT

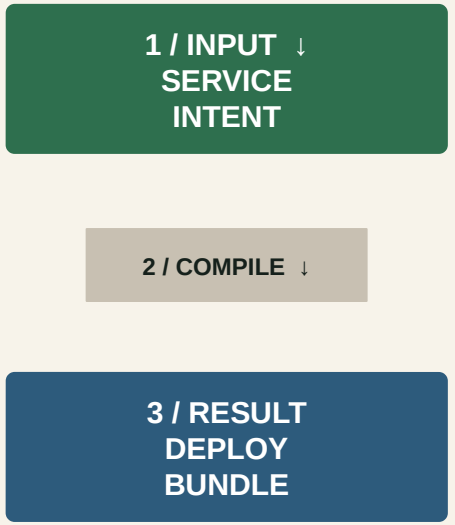
"Prepare LMA, GMA, and pipeline-service for deployment."

WHAT DAGENTS DOES

- Read the service catalog and runtime topology.
- Compile workload settings and environment values.
- Return manifests for the required services.

RESULT

Deployable manifest bundle plus topology information



WHY IT MATTERS The framework links typed plans to the services that run them.

TASK 14 / APPS

Some tasks you can do...

Let a product backend consume DAgents without importing its internals.

- **Consumer app integration:** Use stable APIs and contracts from any backend stack.

TRY IT

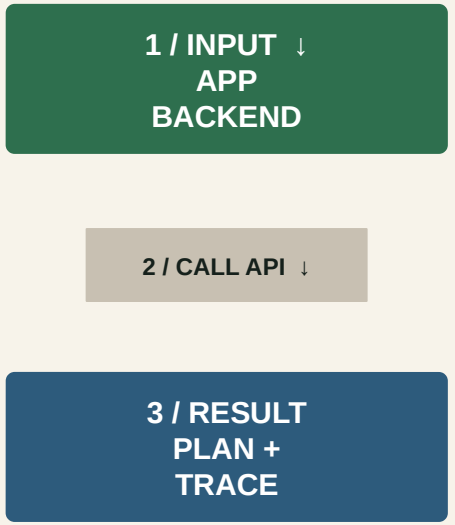
"Add DAgents planning to NL2SQL without moving domain logic."

WHAT DAGENTS DOES

- Call REST, gRPC, or the internal process boundary.
- Send shared source, model, pipeline, or workload contracts.
- Consume plans, run status, SQL, or trace output.

RESULT

A stable integration with a visible DAgents trace



WHY IT MATTERS The product keeps its domain logic while DAgents supplies computation services.

TASK 15 / EXTEND

Some tasks you can do...

Add new infrastructure behind contracts without changing the framework core.

- **New adapters and integrations:** Grow from local demos into cloud and broker-backed deployments.

TRY IT

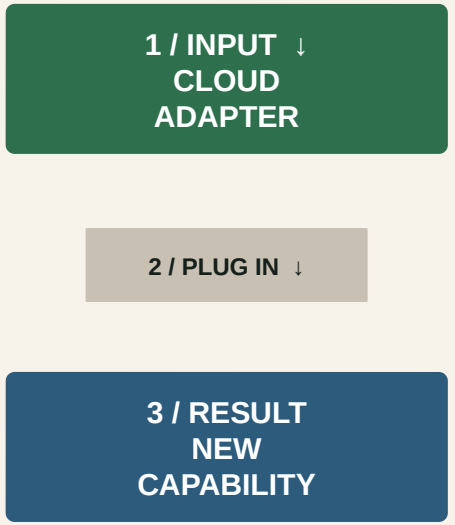
"Add an S3 source adapter and a broker-backed run publisher."

WHAT DAGENTS DOES

- Implement the existing adapter contract.
- Register the new source or publisher capability.
- Verify conformance and deploy independently.

RESULT

Portable capability with no domain-layer rewrite

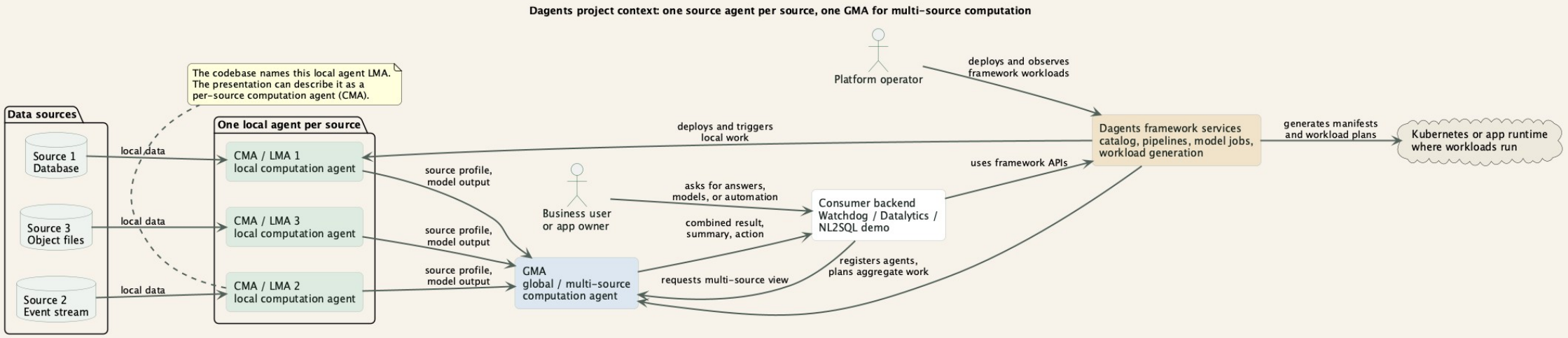


WHY IT MATTERS The same contracts can grow into GCP, AWS, Supabase, and broker integrations.

CONTEXT

Non-technical view: local computation first, combined computation second.

The framework sits between consumer apps, data sources, operators, and the runtime where workloads are deployed.



FRAMEWORK

Dagents is not one product backend. It is a reusable service framework.

Consumer apps plug into it when they need agent orchestration, ML workflow services, or workload generation.

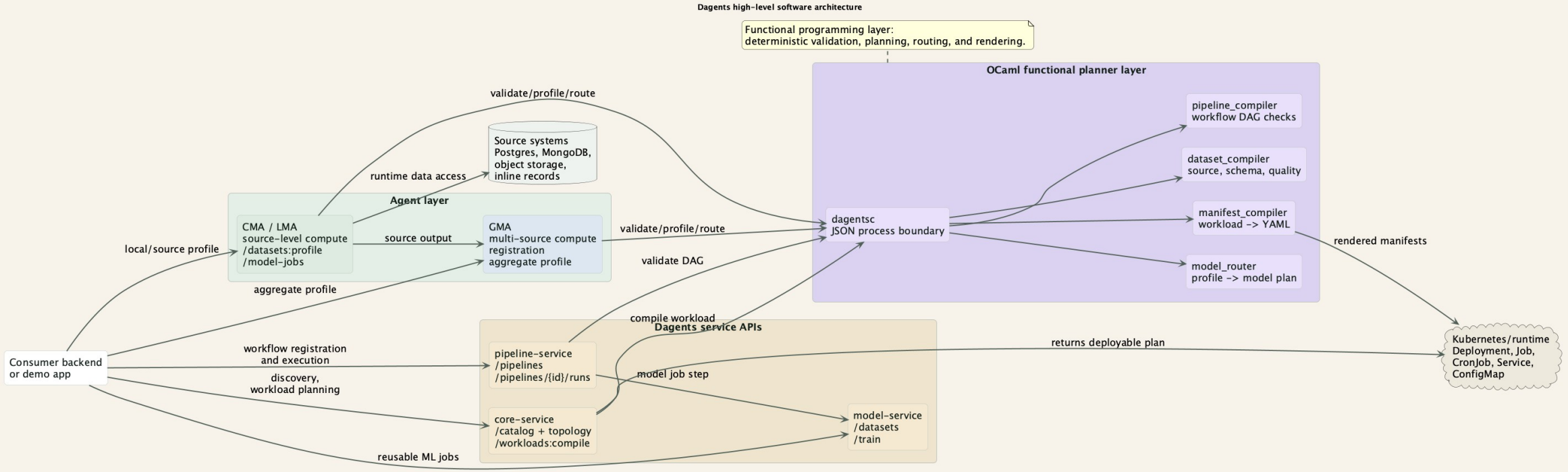
Core service	Pipeline service	Model service	LMA / CMA	GMA
Catalog, topology, workload compilation, and Kubernetes manifest generation.	Register JSON workflows, validate them, and run them against payloads.	Python-first training, checks, benchmark data, and reusable ML jobs.	Source-scoped dataset profiling and local model execution.	Agent registration, aggregation, dispatch tracking, and multi-source profiles.

The important idea: backend teams call stable service APIs instead of rebuilding source profiling, pipeline orchestration, model routing, and manifest generation in every app.

SOFTWARE ARCHITECTURE

High-level software view: runtime services outside, functional planners inside.

The purple block is the OCaml functional programming layer. Services call it through dagentsc.



INTEGRATION

A backend integrates by calling Dagents APIs at the points where it needs planning or execution.

The system is useful because integration is service-based, not a codebase fork.

Architectural points

- LMA owns local/source execution.
- GMA owns aggregate coordination.
- Core service is the deployment facade.
- Pipeline service executes reusable workflows.
- OCaml planners keep validation deterministic.

How to integrate

- Call /api/v1/services or /topology to discover framework services.
- Register a source or pipeline definition.
- Use LMA/GMA /datasets:profile for local or aggregate profiles.
- Use /workloads:compile to turn app components into deployable manifests.
- Read returned plans and traces in your own backend/UI.

Why Dagents is split this way

The framework is intentionally polyglot: each layer owns a clear responsibility.

OCaml for plans

Best for closed choices: source kinds, step kinds, model families.
Pure functions make validation repeatable and easy to test.

Python for runtime

Best for FastAPI, ML libraries, source adapters, and quick experiments.
Training and inference stay close to the Python ecosystem.

Java / Spring for control APIs

Best for typed controllers, validation, configuration, and backend integration.
It provides stable orchestration entrypoints without owning ML execution.

Service boundaries between layers

Spring and Python services call dagentsc through JSON process contracts.
Consumer backends use stable REST or gRPC APIs; failures stay isolated.

“DAgents is not OCaml-first, Python-first, or Java-first. OCaml makes plans precise, Python executes data and models, and Java exposes stable control and integration APIs.”

SERVICE API SCOPE

Core service scope includes workload compilation.

The current FastAPI route shows the intended service contract.

current API example

```
@router.post("/workloads:compile",
response_model=WorkloadPlanResponse)
def compile_workloads(
    request: WorkloadCompileRequest,
    service: ManifestService =
Depends(get_manifest_service),
) -> WorkloadPlanResponse:
    return service.compile(request)
```

consumer call

```
curl -X POST
http://127.0.0.1:8040/api/v1/workloads:compile \
  -H "Content-Type: application/json" \
  -d @docs/demo/inputs/core-workload-compile.json

# returns a stored workload plan plus Kubernetes YAML
# for Deployment, Service, and ConfigMap resources
```

The same pattern exists across the framework: register a workflow, validate a source, profile a dataset, route a model, or compile a workload.

WORK IN PROGRESS

Current implementation scope

The repository contains working slices across agents, services, planners, deployment, and documentation. These boundaries are still being hardened.

Agents

Current: source and aggregate APIs with in-memory state. Next: durable state and broker messaging.

Services

Current: core, pipeline, and model APIs. Next: hardened contracts, authentication, and operations.

Functional planners

Current: typed OCaml planners behind dagentsc. Next: wider contracts and service packaging.

Consumer validation

Current: one demo validates the consumer boundary. Next: Watchdog and Datalytics adapters.

Deployment

Current: Docker Compose and demo scripts. Next: Kubernetes apply and CI/CD.

Documentation

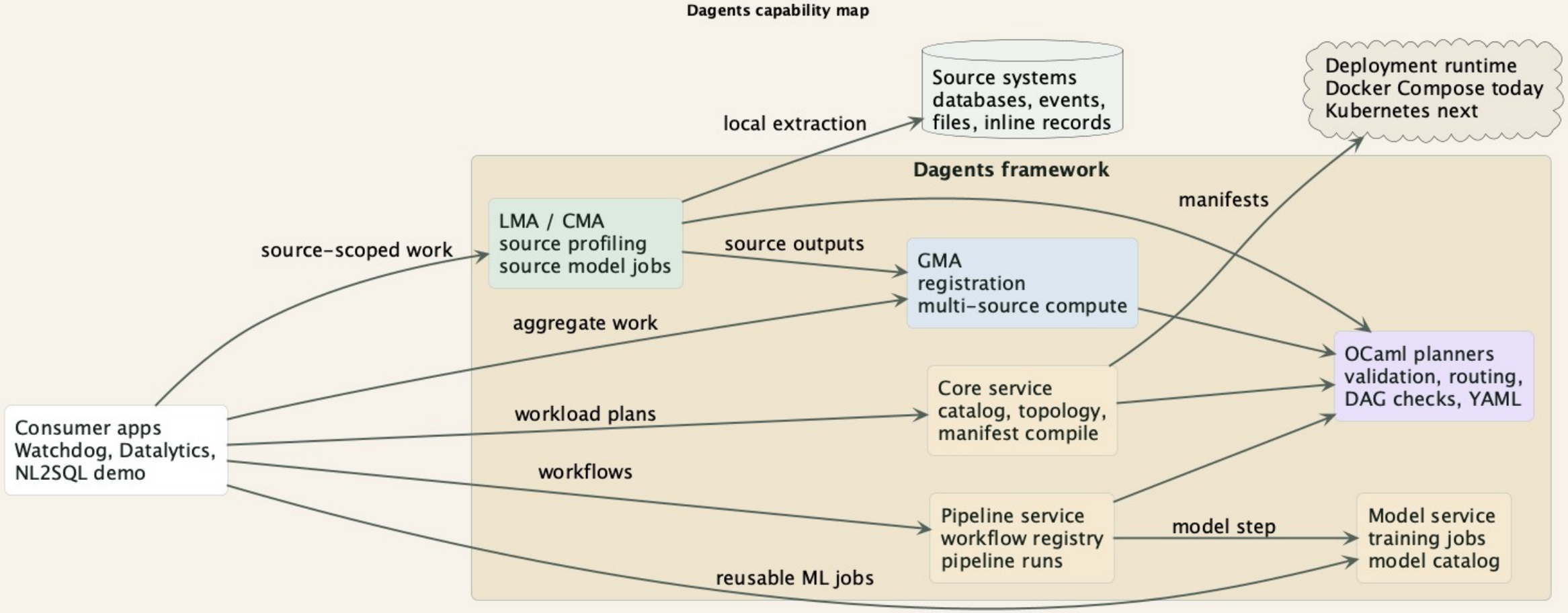
Current: architecture, reports, debug notes, and demo output. Next: operator guides.

Implementation scope, not a production-readiness claim

SCOPE MAP

Current framework scope

This map separates the current implementation boundaries from extension points for future consumer backends.



PlantUML: docs/presentation/puml/dagents_capability_map.puml

AGENT SCOPE

Agent control plane: current work in progress

The current implementation establishes the local-versus-global boundary with in-memory control flows.

Register

GMA records LMA identity, scope, version, and capabilities.

Profile

LMA profiles source records; GMA profiles assimilated records.

Run models

Both agents can accept model-job requests and produce run metadata.

Sync

GMA can plan desired deployments and receive LMA heartbeat/sync state.

This is the first control-plane slice. Persistent state, broker-backed dispatch, and production telemetry remain work in progress.

Current agent API scope: LMA + GMA

SERVICE SCOPE

Service suite: current integration scope

Each service owns a narrow job; contracts and operational hardening remain work in progress.

Service	What it does	Next hardening step
core-service	Catalog, topology, workload compile, Kubernetes YAML	Auth, durable plans, deployment apply
pipeline-service	Registers JSON workflows and runs ordered steps	Persistence, retries, scheduling
model-service	Dataset catalog, training jobs, model-job visibility	Artifact registry and worker queues
LMA / GMA	Source-level and aggregate agent computation	Broker dispatch and durable state
dagentsc	OCaml process boundary for deterministic planning	Versioned contracts and packaging

Integration stays API-based while the contracts mature

POTENTIAL

Where this project can grow

Dagents can become the shared agent layer for data-heavy applications.

Reusable agent layer

Apps can reuse LMA/GMA behavior instead of writing one-off orchestration code.

Traceable decisions

Every request can show checks, plans, routes, and service calls.

Deployment bridge

Typed workload specs can become Docker, Kubernetes, or cloud deployment artifacts.

Safer AI workflows

Policy checks can run before data access, model calls, and generated actions.

PYTHON / RUNTIME

Python: why we chose it and what it does

Python runs the parts of DAgents that talk to data, services, files, models, and external systems.

- Why we chose it: FastAPI makes APIs quick to build, Pydantic checks JSON requests, and PyTorch and data libraries support ML work.
- What it does: Python runs LMA and GMA APIs, pipeline execution, model training and inference, source adapters, and the core service.
- How it is used: A request enters a Python service. The service performs I/O or ML work and calls dagentsc when it needs a typed plan.
- What stays outside: Deterministic compiler rules stay in OCaml. Spring provides a separate control API and integration surface.

Python's job

APIs + I/O
pipelines
ML execution
state

FastAPI
PyTorch

PYTHON MODULES

Python services execute the framework's live workloads

Each service owns an API or runtime concern; shared contracts keep them connected.

Area	What it owns	Why Python fits
LMA / GMA	Source and aggregate agent APIs, runs, state	Fast local and global orchestration
core-service	Catalog, topology, workload facade	HTTP integration and deployment I/O
pipeline-service	Workflow registry and stateful runs	Flexible JSON execution
model-service	Preprocessing, training, evaluation, artifacts	PyTorch and data libraries
NL2SQL backend	Schema, model adapters, DAgents trace	Consumer-specific orchestration

PYTHON REQUEST FLOW

Python turns valid requests into runtime work

The runtime layer coordinates side effects while typed planner boundaries stay explicit.

RECEIVE

FastAPI parses and validates a Pydantic request.

PLAN

The service calls dagentsc or another typed planning boundary.

EXECUTE

It reads data and runs an agent, pipeline, or model action.

RETURN

It stores status and returns output with an execution trace.

Python is the live execution layer; planning remains deterministic and replaceable.

Pytest checks Python runtime behavior

FastAPI clients and dependency boundaries keep most tests quick and focused.

1. Typed fixtures

Datasets, sources, workloads

2. Pytest starts app

FastAPI test clients

3. Service flow runs

Repositories and adapters

4. Integration checked

PostgreSQL or Docker when needed

WHAT WE VERIFY

- LMA and GMA control flow
- Manifest endpoints
- Pipeline step ordering
- Model preprocessing
- NL2SQL model and trace behavior
- PostgreSQL adapters
- dagentsc process calls

```
pytest -q
pytest dagents/tests
pytest services/core-service/tests
pytest services/nl2sql-demo/tests
```

Two kinds of confidence

Python tests prove API and runtime behavior. OCaml tests prove deterministic planner behavior.

JAVA / SPRING

Java / Spring: control APIs and integration

Java provides a strongly typed service layer for control APIs, validation, configuration, and backend integration.

- Why we chose it: Spring Boot provides mature controllers, dependency injection, configuration, validation, testing, and integration tools.
- What it does: Java exposes agent registration, heartbeat, source, deployment, telemetry, catalog, topology, and workload APIs.
- How it is used: Controllers receive requests, application services run the use case, repositories keep state, and gateways call dagentsc.
- What stays outside: Java does not train ML models or replace OCaml planners. Python handles ML; OCaml handles deterministic planning.

Java's job

validate
coordinate
control APIs
integration

Spring Boot
Java 21

JAVA MODULES

Spring services provide a second runtime surface

Controllers stay thin while application services and gateways own coordination.

Area	What it owns	Why it matters
spring-common	Shared request and response records	Keeps service contracts aligned
control API	Registration, heartbeat, source, deployment, telemetry	Stable orchestration entrypoints
control application	Coordination and in-memory repositories	Separates HTTP from state
core API	Catalog, topology, workload compile	Deployment-facing facade
compiler gateway	dagentsc process call and Java fallback	Typed planner without direct FFI

JAVA REQUEST FLOW

Spring keeps request boundaries clear

The control layer keeps transport, coordination, state, and compilation separate.

RECEIVE

A Spring controller validates the incoming request.

COORDINATE

An application service selects and runs the use case.

PERSIST

Repositories update agents, sources, deployments, or runs.

COMPILE + RETURN

The OCaml gateway or Java fallback produces the response.

Spring adds an enterprise integration surface without moving ML execution into Java.

JUnit and MockMvc verify Spring services

Maven runs controller, application, repository, and compiler-gateway tests together.

1. Request fixture

Typed Java records

2. MockMvc calls API

Validation and status codes

3. Service runs

Use cases and repositories

4. Compiler checked

Process call and fallback

WHAT WE VERIFY

- Controller validation
- Agent registration
- Heartbeat and source catalog
- Deployment planning and sync
- Core catalog and topology
- Java manifest compiler
- Process gateway fallback

```
cd services/spring-services
mvn test
mvn package
```

Why the fallback is tested

The service can still return a valid manifest plan when the OCaml executable is unavailable.

OCAML / PLANNING

OCaml: why we chose it and what it does

OCaml handles small planning kernels where invalid states should be rejected before runtime work begins.

- Why we chose it: Variant types, pattern matching, immutable data, and pure functions make rules explicit and easy to test.
- What it does: OCaml validates source schemas, compiles pipeline DAGs, routes model choices, plans workloads, and renders manifests.
- How it is used: Services send JSON to dagentsc and receive a validation report, execution plan, model route, or manifest bundle.
- What stays outside: OCaml is not the API server, database layer, or ML runtime. It plans and validates; Python and Java run services.



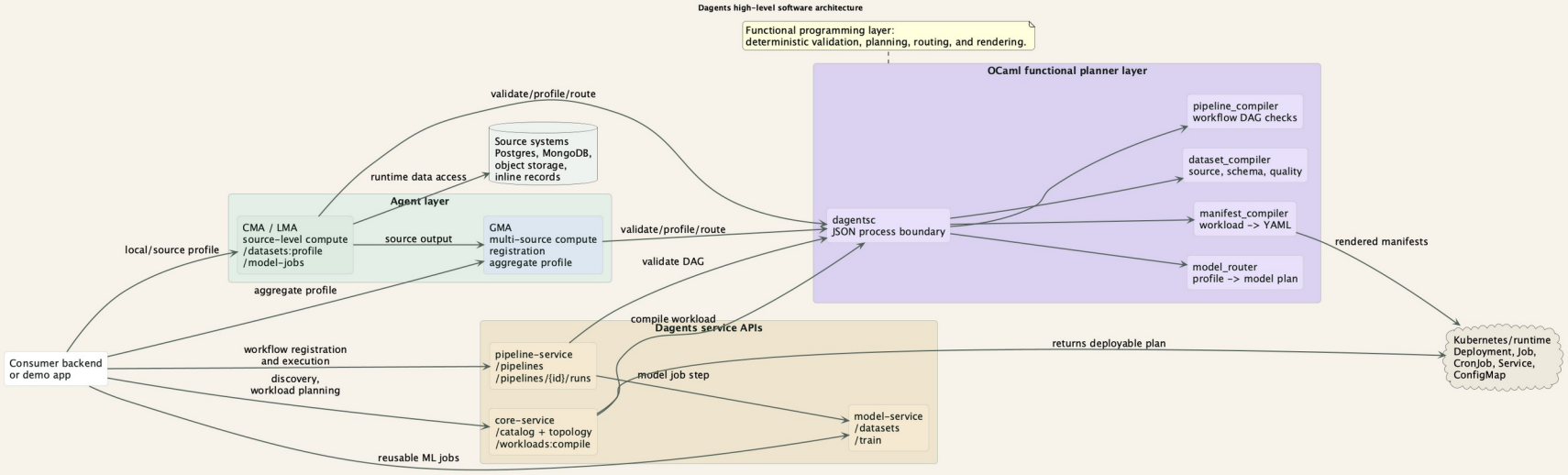
HIGHLIGHTED LAYER

In the high-level architecture, the OCaml modules are the purple planning layer.

They are called by services when a request needs deterministic validation or planning.

OCaml modules

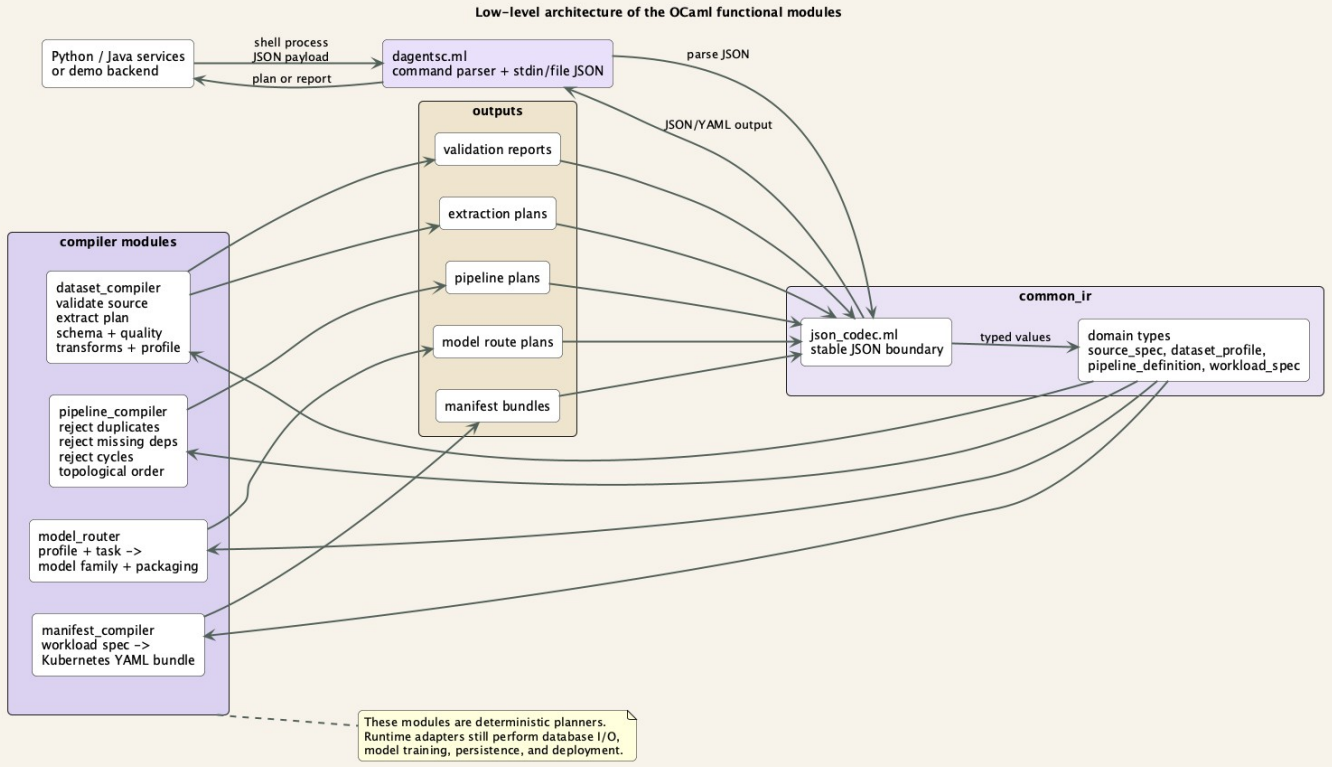
- common_ir gives one typed language for all planners.
- dataset_compiler checks sources and schema work.
- pipeline_compiler validates workflow graphs.
- model_router standardizes model selection policy.
- manifest_compiler renders Kubernetes workload plans.



LOW-LEVEL OCAML

The OCaml layer is a compiler stack over shared typed contracts.

The diagram shows how services enter through dagentsc and leave with reports, plans, routes, or manifests.



- common_ir owns the domain shapes and JSON codecs.
- Each compiler module is a pure planning kernel.
- Outputs are explicit artifacts that runtime services can execute.
- Invalid inputs fail before the runtime does expensive work.

What each planner module does

Each module has a narrow contract: typed input in, explicit artifact out.

Module	Input it understands	Output it returns	Used when
dagentsc.ml	CLI args + JSON	JSON report, plan, route, or YAML	Python/Java services need a planner call
common_ir + json_codec	Source, pipeline, profile, workload JSON	Typed records and variants	Every planner needs the same domain language
dataset_compiler	Sources, records, schema rules, transforms	Profiles, extraction plans, quality reports	Before LMA/GMA/model/pipeline data work
pipeline_compiler	Workflow DAG with dependencies	Ordered steps or validation failure	Before pipeline registration or execution
model_router	Dataset profile + task type	Candidate models + packaging mode	Before model job selection
manifest_compiler	Workload spec	Kubernetes YAML bundle	Core-service workload compilation

These are not long-running ML services. They are deterministic planning kernels that runtime services call when needed.

Dune runs the OCaml workspace tests

Dune is the OCaml build and test tool: it compiles libraries, builds dagentsc, and runs test executables.

Typed test data

records, sources, pipelines

Dune builds modules

common_ir + compilers

Test executable runs

assertions fail fast

CLI build checked

dagentsc.exe for services

What the tests cover

- source validation + invalid source rejection
- time-window and hash partition planning
- schema contracts and quality reports
- record transforms and output schema checks
- pipeline duplicate, missing dependency, and cycle checks
- model routing and packaging modes
- manifest rendering and JSON codec round trips

```
cd bindings/ocaml
opam exec -- dune test

opam exec -- dune build
./bin/dagentsc.exe
```

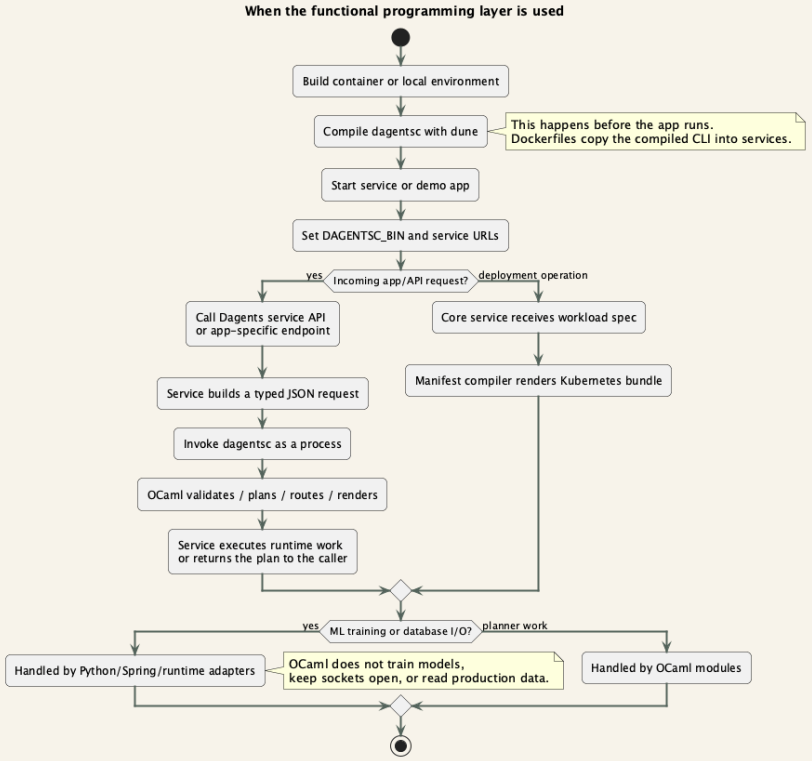
Why this matters for the presentation

Most OCaml tests do not need Docker, databases, or GPUs because the modules are pure planners. That makes the functional layer easy to prove live.

WHEN IT RUNS

The OCaml modules are not project ML APIs. They are planner calls used before and during runtime.

They can run before the app starts as a build artifact, and during app requests when a service needs a plan.

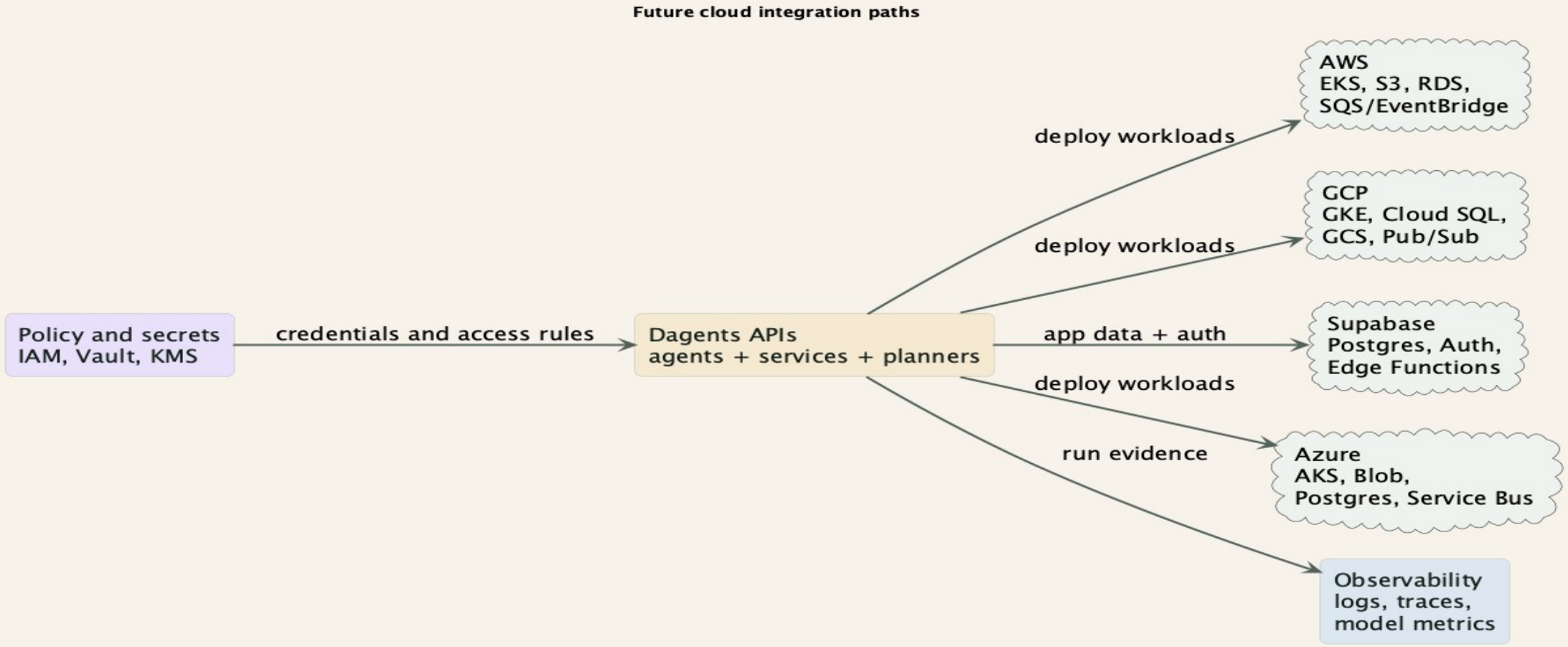


- Before running: Dockerfiles and scripts build dagentsc with dune.
- At startup: services receive DAGENTSC_BIN and service URLs.
- During requests: services invoke dagentsc for validation, routing, DAG planning, or manifest rendering.
- Runtime work still belongs to Python/Spring: reading data, training models, serving APIs, and storing results.

FUTURE

Cloud and backend integrations

The framework is service-based, so it can connect to cloud services without changing the core agent model.



Examples: GCP, AWS, Supabase, Azure

FUTURE

Concrete integration backlog

These are practical next steps after the current local/demo stack.

GCP

GKE for workloads, Cloud SQL for sources, GCS for artifacts, Pub/Sub for events.

AWS

EKS for workloads, S3 for artifacts, RDS for sources, SQS/EventBridge for messaging.

Supabase

Postgres and Auth for quick product integrations; Edge Functions for lightweight hooks.

Observability

OpenTelemetry traces, run metrics, model-job metrics, and audit logs.

Secrets

Vault/KMS/IAM integration so source credentials are never hard-coded.

Persistence

Move in-memory registries to Postgres or a managed store.

Future integrations should keep the same LMA/GMA contracts

FUTURE

Ethical firewalls for AI-era systems

A GRAILS-style layer can be treated as a policy firewall around agent actions.

What it checks

Should this request access this data? Should this model be called? Should this output be released?

Where it runs

Before source extraction, before model routing, before workload deployment, and before returning generated output.

What it records

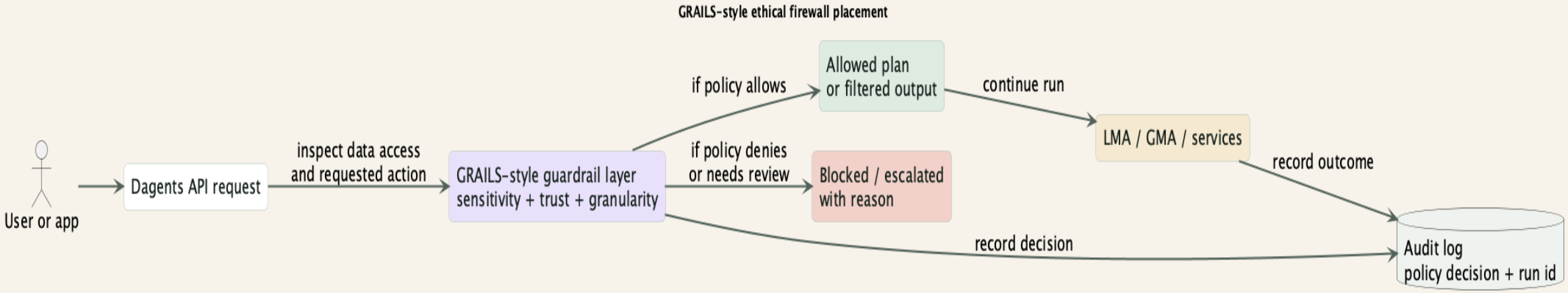
The policy decision, reason, request id, agent id, data scope, and final action.

This would make Dagents useful not only for computation, but also for responsible control of computation. The idea fits the LMA/GMA model because local and global decisions can have different policy rules.

FUTURE

Where a GRAILS-style guardrail fits

The guardrail should be a first-class service boundary, not a UI-only warning.



Policy can stop a run, narrow the data scope, or require human review.
Audit logs should connect policy decisions with Dagents run ids and deployment plans.
This can support college research around ethical firewalls while staying practical for backend teams.

ROADMAP

Next build steps

The path forward is incremental: harden the current stack, then add cloud and policy boundaries.

Phase	Goal	Output
1. Stabilize	Persist registrations and runs; improve error reporting.	Reliable local demo and repeatable tests.
2. Deploy	Move workloads from Compose to Kubernetes.	Core-service manifests applied to a real cluster.
3. Integrate	Connect GCP/AWS/Supabase data and auth services.	Reusable adapters and secret handling.
4. Govern	Add GRAILS-style policy checks and audit logs.	Ethical firewall for data, models, and outputs.
5. Scale	Broker-backed messaging and distributed agent execution.	Production-ready multi-source computation.

CLOSE

Dagents gives consumer backends reusable agent orchestration plus deterministic planning.

The project story is simple: compute locally, combine globally, plan with typed functional modules, and deploy through services.

- CMA / LMA per source keeps local computation close to the data.
- GMA combines many local outputs into multi-source computation.
- Framework services expose catalog, pipeline, model, agent, and workload APIs.
- OCaml planners make validation and planning predictable.
- NL2SQL proves a real app can plug into the framework without becoming the framework.

Appendix

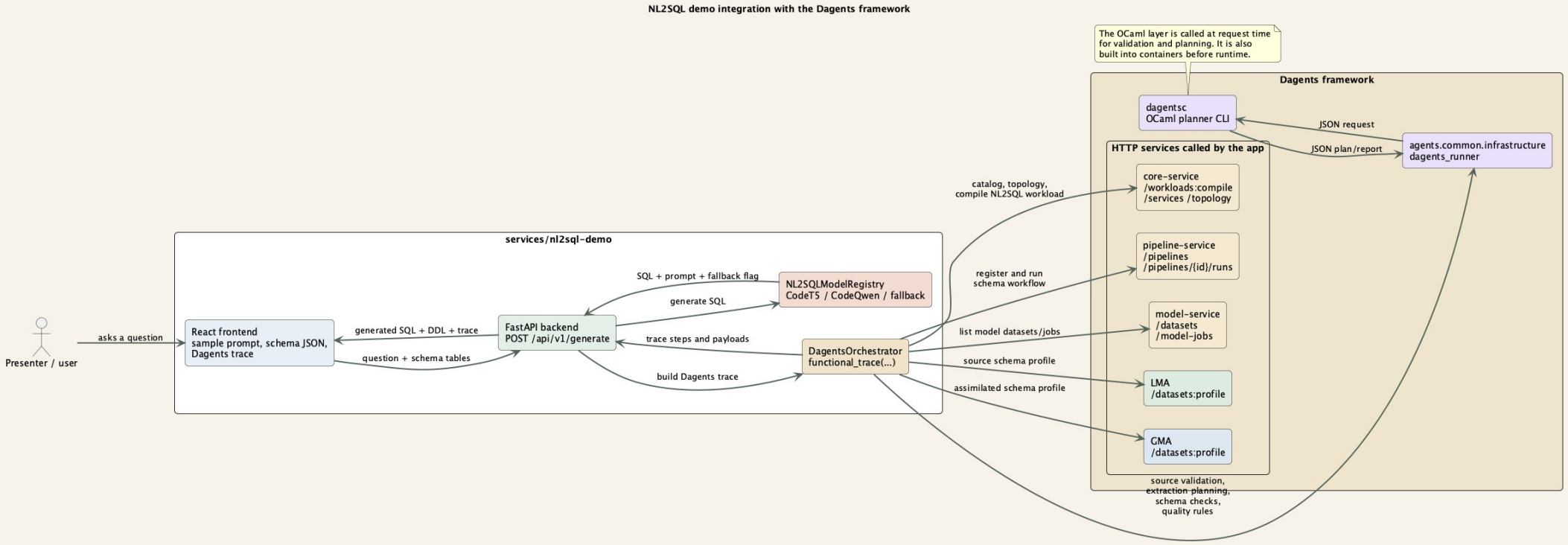
NL2SQL demo and TypeScript UI

Architecture, integration flow, frontend layer, and demo runbook

NL2SQL INTEGRATION

The demo app plugs into Dagents through a thin backend orchestration layer.

The app still owns the UI and SQL generation. Dagents owns traceable planning, validation, service checks, and workload compilation.



BACKEND WIRING

NL2SQL calls Dagents while handling a normal /generate request.

The trace returned to the UI is built from actual planner and service calls.

services/nl2sql-demo/backend/app/services/dagents_orchestrator.py

```
self._append_trace(trace, "Dagents SourceSpec validation",
    lambda: validate_dataset_source(inline_source))
self._append_trace(trace, "Dagents extraction planning",
    lambda: compile_dataset_extraction(inline_source))
self._append_trace(trace, "Dagents pipeline DAG planning",
    lambda: self._pipeline_plan_payload())
self._append_trace(trace, "core-service workload compile",
    lambda: self._compile_demo_workload())
```

- The schema is converted into records for validation and profiling.
- dagentsc plans the schema and pipeline parts through JSON.
- Core, pipeline, model, LMA, and GMA services are touched in the trace.
- The SQL model adapter remains app-specific.

NL2SQL uses app adapters plus Dagents service calls

The app owns SQL generation; Dagents owns traceable planning, validation, and orchestration.

/api/v1/models
list adapters

/api/v1/generate
schema + question

adapter selected
CodeT5 / CodeQwen / fallback

SQL + trace
prompt, DDL, fallback detail

Adapter	What it loads	Demo behavior
CodeT5 seq2seq	Zip artifact with config and weights	Uses question/context DDL prompt for compact text-to-SQL inference.
CodeQwen LoRA	PEFT adapter over CodeQwen chat model	Uses system + user chat prompt; needs heavier runtime resources.
heuristic_demo	No model artifact	Deterministic fallback; response includes fallback_detail for clarity.

Dagents APIs touched during generation

core /services + /topology + /workloads:compile | pipeline /pipelines + /runs | LMA/GMA /datasets:profile + /model-jobs | GMA registration + heartbeat

NL2SQL integration: current demo scope

The demo exercises the current planning and service path around SQL generation.

Dagents trace

The backend returns planner checks, service calls, LMA/GMA actions, and workload compilation evidence.

Model adapter

CodeT5 model artifacts are discovered from models/*.zip and loaded by the adapter when dependencies are present.

Demo scripts

run_compose_demo.sh, probe_api.sh, and stop_compose_demo.sh make the presentation path repeatable.

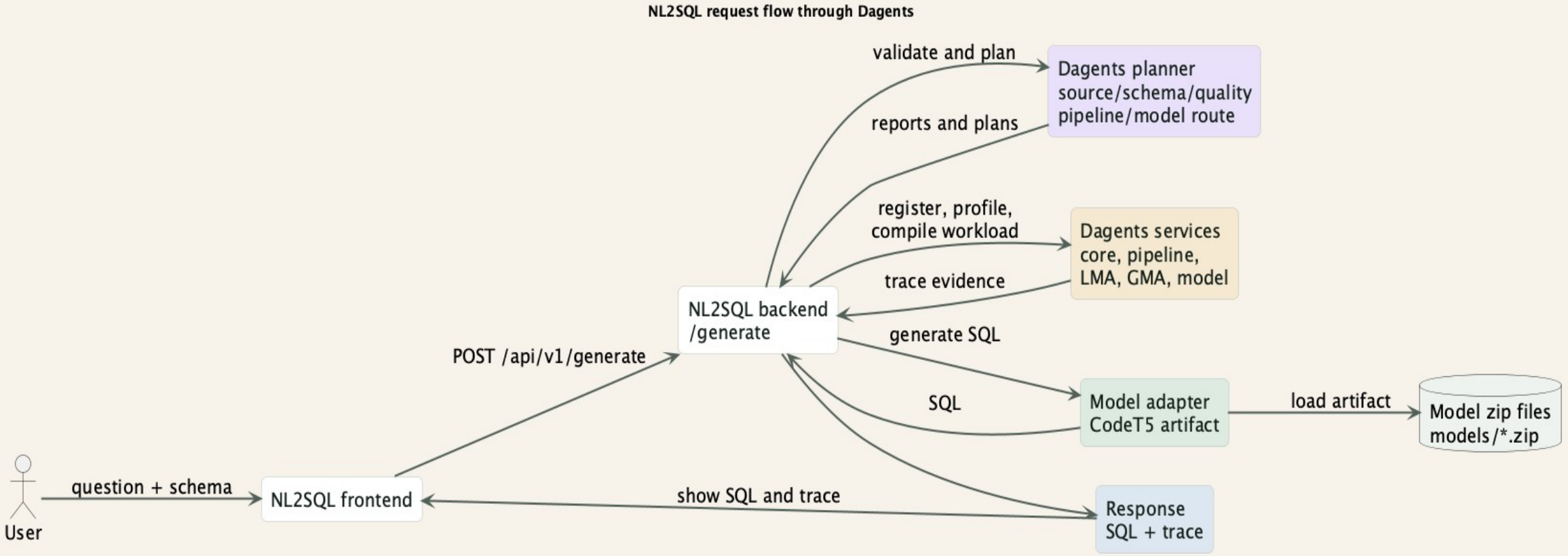
The current probe checks 22 DAgents trace steps.
The model adapter can use the packaged artifact when dependencies are available.
The deterministic fallback remains available for low-resource machines.

Current demo evidence: docs/demo/expected/nl2sql-generate.json

DEMO FLOW

What happens during /generate

The app request becomes a traceable sequence of planner calls, service calls, and model generation.



PlantUML: docs/presentation/puml/dagents_request_flow.puml

TYPESCRIPT / UI

TypeScript / React: the NL2SQL interface

TypeScript owns the NL2SQL browser experience and keeps user-interface state and API responses typed.

- Why we chose it: TypeScript catches UI mistakes during builds, React fits interactive forms, and Vite provides a fast build loop.
- What it does: The UI captures the question, schema, and sample data, calls the backend, and displays SQL with the DAgents trace.
- How it is used: React stores input and response state, sends a typed request to /api/v1/generate, and renders the returned result.
- What stays outside: The browser does not run agents, planners, models, or database queries. Sensitive work stays in backend services.

TypeScript's job

input
backend calls
render SQL
show trace

UI state

React 19
Vite

The frontend makes framework results usable

The UI is deliberately small and keeps the backend contract visible.

Area	What it owns	Why it matters
App.tsx	Prompt, schema, sample, response state	Main demo experience
main.tsx	React application bootstrap	Small composition root
API request	POST /api/v1/generate	Explicit UI and backend contract
Trace view	Planner steps, calls, fallback details	Makes DAgents explainable
Vite + Docker	Dev server and production build	Repeatable local demo

TYPESCRIPT USER FLOW

NL2SQL keeps the demo path short

One screen shows the user input, generated SQL, and the framework trace.

EDIT INPUT

The user enters a question, schema, and optional sample.

SUBMIT

The browser posts a generate request to the backend.

RECEIVE

The API returns SQL, trace steps, and model status.

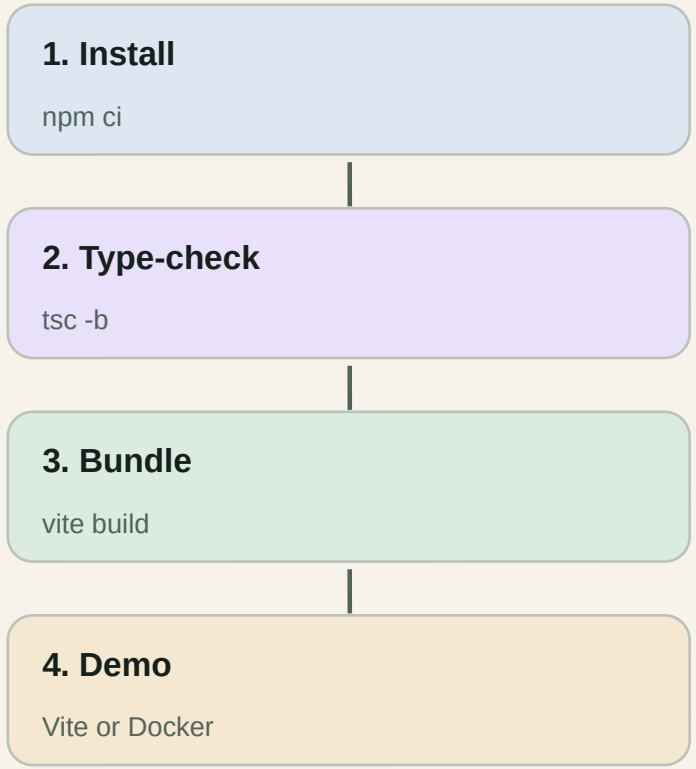
RENDER

React displays the SQL and explains how DAgents produced it.

The frontend demonstrates framework use without duplicating backend intelligence.

TypeScript builds catch UI errors early

The frontend is checked through TypeScript compilation, Vite bundling, and browser smoke tests.



WHAT WE VERIFY

- React entrypoint compiles
- Prompt and schema state is typed
- API responses are handled
- Production assets are emitted
- Frontend reaches the backend
- SQL and trace remain readable

```
cd services/nl2sql-demo/frontend
npm ci
npm run build
npm run dev
```

A clear responsibility boundary

The UI proves the framework is consumable. It does not duplicate planner or model logic.

DEMO VALUE

In NL2SQL, the functional layer makes the demo safer and easier to explain.

The UI can show exactly what Dagents checked and planned before returning SQL.

Useful for schema input

Source validation, extraction planning, schema contract checks, and quality rules run over the table metadata.

Useful for workflow safety

Pipeline DAG planning rejects invalid graph structure before a workflow is executed.

Useful for deployment

Core service compiles the NL2SQL backend into a workload plan and manifest bundle.

Useful for traceability

The response includes planner and service trace steps, so the demo can show what happened.

Short answer for Q&A: OCaml is called as needed by the app/services. It is not the SQL model and not a separate ML server.

DEVELOPMENT

Challenges and steering decisions

We used the AI assistant as an implementation partner, but kept decisions grounded in test results and logs.

Challenge	How we steered it	State change
Docker model deps	Use logs, identify CUDA wheel blow-up, pin CPU-friendly Torch line.	Backend image now builds with model deps.
Fallback SQL	Trace the exception instead of hiding it.	Model adapter now loads CodeT5 artifact.
JSON key casing	Compare trace payloads and find false schema failures.	Record field names are preserved.
OCaml readability	Ask for modular common_ir without changing behavior.	Types are grouped while old API remains.
Demo reliability	Make scripts executable and probe real service behavior.	Probe validates trace and model path.

This slide also answers the development-challenges report request

DEMO RUNBOOK

Run the demo with one terminal for the app and one terminal for the probe.

Use local mode for speed. Use compose mode when you want to show the whole Dagents stack.

local mode

```
# Fast local demo
services/nl2sql-demo/scripts/run_local_demo.sh

# Probe the backend and save the response
services/nl2sql-demo/scripts/probe_api.sh
```

compose and functional demo

```
# Full Dagents stack with NL2SQL
services/nl2sql-demo/scripts/run_compose_demo.sh

# Stop containers
services/nl2sql-demo/scripts/stop_compose_demo.sh

# Functional layer output screenshots
docs/demo/run_functional_layer_demo.sh
```

Open http://127.0.0.1:5173